

RAPPORT DE STAGE : MISSION ASSISTANT INGÉNIEUR

Conception, Benchmark et Déploiement d'un Simulateur Vocal d'Intelligence Artificielle pour l'Entraînement des Conseillers Bancaires de la BMCI

Organisme d'accueil : Banque Marocaine pour le Commerce et l'Industrie (BMCI), Groupe BNP Paribas

Département d'accueil : Département de la Formation et de la Transformation Digitale

Institution : École Centrale Casablanca (ECC)

Auteur : Élève Ingénieur de Centrale Casablanca

Période de stage : 2024

Sommaire

1. Introduction, Contexte et Formulation du Problème

- 1.1. Contexte industriel de la BMCI
- 1.2. Problématique et besoins de formation des conseillers clientèle
- 1.3. Enjeux opérationnels et objectifs du simulateur vocal

2. Cartographie et Étude Comparative de l'État de l'Art

- 2.1. Concepts métriques fondamentaux : WER, CER, RTFx et MOS
- 2.2. Cartographie et benchmark des modèles ASR / STT
- 2.3. Benchmark des modèles de traitement du langage (LLM) en français
- 2.4. Étude comparative des modèles de synthèse vocale (TTS)

3. R&D : Fine-Tuning de Modèles Locaux (SLM) et Plateforme de Simulation

- 3.1. Constitution du jeu de données (dataset) bancaire français
- 3.2. Expérimentations de Fine-Tuning LoRA sur architectures légères (Qwen, SmolLM, TinyLlama)
- 3.3. Développement de l'interface Streamlit et intégration des Guardrails
- 3.4. Résolution des contraintes matérielles (GPU Intel vs Nvidia CUDA, Colab & Kaggle)

4. Architecture de la Solution Temps Réel (LiveKit & WebRTC)

- 4.1. Choix technologique de LiveKit et du protocole WebRTC
- 4.2. Flux de données asynchrone de la pipeline voix-à-voix
- 4.3. Gestion du dialogue naturel : Full-Duplex, Barge-in et Endpointing VAD

5. Résolution des Verrous Techniques Majeurs

- 5.1. Résolution du crash WebRTC par rééchantillonnage dynamique (24kHz à 48kHz)

- 5.2. Conception du superviseur anti-crash de résilience (`run_agent.py`)
- 5.3. Implémentation du mécanisme de secours (fallback) STT face aux erreurs 429
- 5.4. Ingénierie de prompt avancée : Modération et identification progressive (Mme. Sarah Bennani)

6. Analyse Comparative : L'Évolution Speech-to-Speech (Moshi) et Perspectives

- 6.1. Analyse des contraintes d'infrastructure et de transport de Moshi (Kyutai)
- 6.2. Perspectives d'intégration RAG et classification émotionnelle acoustique

7. Bibliographie

1. Introduction, Contexte et Formulation du Problème

1.1. Contexte industriel de la BMCI

La Banque Marocaine pour le Commerce et l'Industrie (BMCI), filiale du groupe international BNP Paribas, est un acteur de premier plan dans le paysage bancaire marocain. Avec un réseau étendu d'agences et des centres de relation client à forte activité, la BMCI place l'excellence opérationnelle et la satisfaction client au cœur de ses priorités stratégiques. La digitalisation croissante des services financiers a modifié les attentes des clients : ceux-ci recherchent une réactivité immédiate et une haute compétence technique lorsqu'ils s'adressent à un conseiller.

1.2. Problématique et besoins de formation des conseillers clientèle

Les conseillers de clientèle, qu'ils travaillent en agence physique ou en centre d'appels, sont régulièrement confrontés à des clients mécontents, paniqués ou impatientes. La gestion de ces situations complexes requiert non seulement une maîtrise parfaite des procédures bancaires, mais également des compétences relationnelles (soft skills) solides comme l'empathie, la gestion du stress, et la capacité de négociation.

Traditionnellement, la formation à ces situations conflictuelles s'effectue via des simulations de jeux de rôle (roleplay) animées par des formateurs ou des managers. Cette méthode humaine présente toutefois des limites de passage à l'échelle :

- **Disponibilité limitée** : Les séances nécessitent un encadrement un-à-un coûteux en temps pour les formateurs.
- **Manque de standardisation** : Deux formateurs différents évalueront différemment les réactions d'un apprenant.
- **Absence d'entraînement autonome** : Le conseiller ne peut pas s'exercer de manière indépendante lorsqu'il en ressent le besoin.

1.3. Enjeux opérationnels et objectifs du simulateur vocal

Pour répondre à ces limites, ce projet de R&D vise à développer un **simulateur vocal basé sur l'intelligence artificielle**. L'objectif est de mettre en situation professionnelle l'apprenant en simulant un appel téléphonique avec un client virtuel mécontent.

Sur le plan technique, la simulation doit s'approcher au maximum du réalisme d'une véritable conversation. L'agent virtuel doit :

- **Répondre en moins d'une seconde** pour éviter tout sentiment d'interaction robotique.
 - **Exprimer des émotions adaptatives** (irritation, frustration, puis soulagement si la posture du conseiller est adéquate).
 - **Détecter les interruptions (Barge-in)** afin de permettre un échange spontané.
 - **Respecter les contraintes de sécurité bancaire (Kyc)** en n'accordant des informations personnelles ou des transactions qu'après une procédure d'identification rigoureuse.
-

2. Cartographie et Étude Comparative de l'État de l'Art

2.1. Concepts métriques fondamentaux : WER, CER, RTFx et MOS

Afin de classer et de comparer les différents modèles existant sur le marché de l'intelligence artificielle vocale et linguistique, nous avons utilisé des métriques scientifiques standardisées :

- **WER (Word Error Rate)** : Taux d'erreur sur les mots. Il mesure le nombre d'insertions, de substitutions et de délétions de mots nécessaires pour aligner la transcription du modèle sur la transcription de référence. Plus le WER est bas, plus le modèle de reconnaissance vocale est précis.
 - **CER (Character Error Rate)** : Taux d'erreur sur les caractères, particulièrement utile pour évaluer la précision sur les acronymes ou les suites de chiffres (comme les identifiants bancaires ou de carte d'identité).
 - **RTFx (Real-Time Factor)** : Facteur de temps réel. Il correspond au rapport entre la durée du fichier audio et le temps nécessaire au modèle pour le traiter. Un RTFx supérieur à un indique que le traitement s'effectue plus vite que le temps réel.
 - **MOS (Mean Opinion Score)** : Score d'opinion moyen. Évalué par des utilisateurs ou par des modèles d'évaluation de la qualité de la voix (comme UTMOS), il attribue une note de 1 (très artificiel) à 5 (voix humaine parfaite) pour qualifier le naturel d'une voix synthétique.
-

2.2. Cartographie et benchmark des modèles ASR / STT

L'état de l'art de la reconnaissance vocale en français (Speech-to-Text) a été cartographié à l'aide de tests sur des jeux de données publics francophones de référence (datasets CoVoST, MLS, Fleurs).

! [Figure 1: Graphique de la cartographie des performances ASR sur Hugging Face (WER vs RTFx)] (file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1781536096829.png)

L'étude des modèles open-source et propriétaires met en évidence des différences significatives de précision et de temps de calcul :

- Les solutions d'API propriétaires comme `elevenlabs/scribe_v2` (WER de 2.67%) et `CohereLabs/cohere-transcribe` (WER de 3.83%) offrent les plus hautes précisions. Cohere s'illustre particulièrement par une vitesse d'exécution très élevée (RTFx de 491.36).
- Les modèles locaux open-source comme `openai/whisper-large-v3` (WER de 4.81%) et sa version optimisée `whisper-large-v3-turbo` (WER de 5.56%) représentent la référence pour des déploiements confidentiels en local, bien qu'ils

nécessitent une accélération matérielle (GPU) sous peine d'une vitesse de calcul trop lente sur processeur (CPU) seul.

Impact acoustique du terminal de l'apprenant (Tests réels BMCI)

Dans le cadre du déploiement à la BMCI, nous avons évalué ces modèles sur des fichiers audio de 3 minutes enregistrés dans différentes conditions. Les résultats montrent que :

- Le **téléphone portable** offre la meilleure transcription (WER de 3.12% pour ElevenLabs) grâce à l'efficacité du microphone de proximité et des puces de réduction de bruit ambiant.
 - Le **casque audio** donne d'excellents résultats pour les API cloud (WER de 3.95%), mais affiche des faiblesses avec les modèles locaux en raison d'un signal audio étouffé.
 - Le **micro intégré du PC de bureau** obtient les moins bons résultats (WER de 8.32% avec Cohere), perturbé par la réverbération de la pièce et le bruit continu de la ventilation de la machine.
-

2.3. Benchmark des modèles de traitement du langage (LLM) en français

Pour incarner l'intelligence et la cohérence de l'agent conversationnel, les modèles de langage ont été comparés sur leur capacité à suivre scrupuleusement des instructions de rôle complexes en français (IFEval FR), leur raisonnement (GPQA FR) et leur culture académique (Bac FR).

![Figure 2: Positionnement des modèles de langage francophones sur le leaderboard d'évaluation administrative](file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1782775116530.png)

- **DeepSeek-R1-Distill-Llama-70B** et **Mistral-Large-Instruct-2411** dominent le classement avec un score strict de suivi des consignes de 66% à 67%.
 - Pour des architectures plus légères, adaptées à des déploiements sur serveurs modestes ou locaux, des modèles comme **jpacifico/Chocolatine-2-14B** (score global de 42.24%) ou **Qwen2.5-14B-Instruct** (score global de 41.86%) montrent d'excellentes aptitudes à soutenir un dialogue bancaire cohérent.
-

2.4. Étude comparative des modèles de synthèse vocale (TTS)

La synthèse vocale (Text-to-Speech) est la brique finale qui donne sa voix au client mécontent. Pour notre scénario de jeu de rôle, la voix doit impérativement retransmettre la colère et l'impatience.

![Figure 3: Évaluation du naturel de la voix (MOS) et de la latence de traitement des solutions TTS](file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1781536071838.png)

- **Hume AI (modèle Octave)** obtient le meilleur naturel (MOS de 4.03), mais souffre d'une latence moyenne de génération élevée (1.97 seconde), trop lente pour un échange téléphonique fluide.
- **F5-TTS** (modèle local de clonage) est extrêmement qualitatif pour cloner une voix spécifique à partir d'un échantillon court, mais s'avère inutilisable en conditions réelles

sans accélération GPU dédiée (128 secondes de calcul pour 3 secondes de voix produites sur CPU).

- **Mistral Voxtral** s'est révélé être le meilleur choix pour notre cas d'usage. Il propose une latence moyenne faible (1.57s), un excellent naturel (MOS de 3.78) et possède une voix féminine nativement en colère (`fr_marie_angry`) qui évite d'avoir à coder des instructions d'émotions vocales secondaires.

3. R&D : Fine-Tuning de Modèles Locaux (SLM) et Plateforme de Simulation

3.1. Constitution du jeu de données (dataset) bancaire français

L'une des étapes structurantes du projet a consisté à créer un **jeu de données (dataset) bancaire francophone** pour adapter les modèles de langage à la réalité métier de la BMCI. Nous avons conçu un ensemble de 9 scénarios de simulation d'agence et de centre d'appels :

1. Carte bancaire avalée par un distributeur automatique.
2. Retrait exceptionnel de fonds en espèces au-delà du plafond autorisé.
3. Blocage de l'application bancaire mobile suite à de mauvaises saisies.
4. Contestation de frais de tenue de compte jugés abusifs par le client.
5. Suivi d'un virement de salaire non reçu sur le compte.
6. Demande d'informations réglementaires sur les crédits immobiliers.
7. Réclamation suite à une opération de paiement frauduleuse sur internet.
8. Augmentation permanente du plafond de paiement pour un achat imminent.
9. Contestation d'un chèque débité deux fois par erreur technique.

3.2. Expérimentations de Fine-Tuning LoRA sur architectures légères

Pour faire tourner l'agent conversationnel en local sur les machines de la banque, nous avons testé le fine-tuning de modèles de langage de petite taille (SLM - Small Language Models). Nous avons utilisé la méthode **LoRA (Low-Rank Adaptation)**, qui permet d'ajuster les poids du modèle en n'entraînant qu'une infime partie des paramètres, économisant ainsi la mémoire vidéo requise.

Les entraînements ont ciblé plusieurs architectures de petite taille :

- **TinyLlama-1.1B**
- **SmolLM2-1.7B**
- **Qwen2.5-0.5B** et **Qwen2.5-1.5B**
- **BloomZ** et **CroissantLLM**

Le modèle **Qwen2.5-0.5B-BMCI-Client-Finetune-1** s'est révélé être le plus stable et efficace. Malgré ses 500 millions de paramètres seulement, il a correctement assimilé la posture de client mécontent et le vocabulaire bancaire français sans souffrir d'hallucinations majeures.

3.3. Développement de l'interface Streamlit et intégration des Guardrails

Pour tester et valider ces modèles locaux, nous avons développé une plateforme sous **Streamlit** faisant office de tableau de bord. Cette application permet de :

- Sélectionner et charger à chaud les checkpoints fine-tunés.
- Choisir le scénario de dialogue à lancer.
- Visualiser les métriques de la conversation.

Un enjeu technique critique était d'empêcher les modèles de langage légers de sortir de leur rôle. Les petits modèles souffrent parfois de dérives de rôle : ils oublient qu'ils jouent le client et se mettent à formuler les réponses à la place du conseiller. Pour contrer cela, nous avons implémenté des **Guardrails (garde-fous)** au niveau du code de l'interface. Un analyseur de texte intercepte chaque phrase générée par le modèle avant son affichage et bloque les structures de phrases typiques d'un conseiller bancaire (ex: *"Bonjour, comment puis-je vous aider ?"* ou *"Je vais consulter votre dossier"*), forçant le modèle à ré-échantillonner une réponse compatible avec sa posture de client.

3.4. Résolution des contraintes matérielles

L'entraînement de modèles de langage nécessite des puces graphiques compatibles avec la technologie CUDA de NVIDIA. Or, les postes de travail locaux de la BMCI ne disposent que de processeurs graphiques intégrés Intel, inadaptés au calcul intensif IA.

Pour surmonter cette contrainte matérielle, nous avons déporté l'ensemble de l'architecture d'entraînement (Fine-Tuning LoRA) sur des infrastructures cloud à ressources partagées (**Google Colab** et **Kaggle**). Nous y avons implémenté un système de sauvegarde automatique des checkpoints d'entraînement sur **Google Drive** à la fin de chaque époque. Cette précaution a permis de fiabiliser le processus de R&D en prévenant les pertes de données causées par les déconnexions intempestives des sessions gratuites du cloud.

4. Architecture de la Solution Temps Réel (LiveKit & WebRTC)

4.1. Choix technologique de LiveKit et du protocole WebRTC

La transition de l'application de chat textuel vers une véritable simulation vocale interactive exige une infrastructure réseau adaptée. Nous avons écarté les API classiques basées sur HTTP (qui imposent une latence de chargement trop élevée) ou les WebSockets bruts (complexes à synchroniser pour des flux audio continus).

Nous avons opté pour le framework **LiveKit**, basé sur le protocole de communication en temps réel **WebRTC**. WebRTC offre des caractéristiques de transport idéales :

- Connexion directe à faible latence (UDP) entre l'utilisateur et l'agent vocal.
- Codec audio **Opus** à débit adaptatif, offrant une excellente qualité vocale même sur des connexions réseau instables.
- Gestion native de la suppression d'écho (Acoustic Echo Cancellation) et contrôle automatique du gain.

![Figure 4: Schéma d'architecture technique de la pipeline WebRTC LiveKit] (file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1781796513047.png)

4.2. Flux de données asynchrone de la pipeline voix-à-voix

L'agent conversationnel se connecte au salon LiveKit en tant que participant virtuel. Le flux de données audio circule de manière asynchrone à travers les étapes suivantes :

1. **Écoute active** : LiveKit capture le flux audio du microphone de l'apprenant.
2. **Transcription (STT)** : Dès que le détecteur de parole (Vad) valide une fin de phrase, les paquets audio accumulés en mémoire sous forme de tampon (buffer) sont envoyés à l'API de reconnaissance vocale.
3. **Réflexion (LLM)** : Le texte transcrit est injecté dans le modèle de langage principal avec les consignes de rôle (System Prompt) et l'historique de la discussion.
4. **Synthèse vocale (TTS)** : Les phrases de réponse produites par le LLM sont envoyées au moteur de synthèse vocale pour être transformées en paquets audio.
5. **Diffusion** : LiveKit réinjecte les paquets audio de réponse dans le flux de retour WebRTC de l'apprenant.

4.3. Gestion du dialogue naturel : Full-Duplex, Barge-in et Endpointing VAD

Pour reproduire le confort d'un appel téléphonique naturel, nous avons configuré l'agent en mode **Full-Duplex** avec détection d'interruption (**Barge-in**). Si l'agent vocal est en train de prononcer sa réponse et que l'apprenant l'interrompt en parlant, le détecteur d'activité vocale (VAD) repère la voix humaine. L'agent coupe alors instantanément l'émission de son propre flux audio et vide ses tampons de parole pour écouter l'utilisateur.

Nous avons fixé le paramètre de silence de fin de parole (**min_delay**) à **0,8 seconde**. Ce délai garantit que l'agent n'interrompt pas prématurément l'apprenant si celui-ci marque une courte pause d'hésitation dans sa phrase, tout en maintenant un temps de réaction global sous la seconde après une véritable fin de réplique.

5. Résolution des Verrous Techniques Majeurs

5.1. Résolution du crash WebRTC par rééchantillonnage dynamique

- **Le problème** : Le moteur de synthèse vocale de Mistral Voxtral génère un flux audio échantillonné à une fréquence de **24kHz**. Lorsque ce flux était envoyé directement dans les canaux audio de LiveKit sous un environnement Windows, la bibliothèque native WebRTC écrite en Rust (`webrtc-sys`) paniquait lors de la tentative de désérialisation et d'empaquetage des données, provoquant le crash immédiat et irrémédiable de l'application Python.
 - **La solution** : Nous avons développé un filtre de traitement du signal audio intégré au script de l'agent. Ce module intercepte le flux audio de 24kHz produit par l'API et utilise la classe de rééchantillonnage dynamique (`rtc.AudioResampler`) de LiveKit pour convertir en temps réel les paquets audio vers la fréquence standard WebRTC de **48kHz** (mono). Les données rééchantillonnées sont ensuite envoyées de manière fluide dans le canal audio LiveKit, résolvant définitivement le crash système.
-

5.2. Conception du superviseur anti-crash de résilience (`run_agent.py`)

- **Le problème** : WebRTC gère de nombreuses communications réseau asynchrones en arrière-plan. Lors de déconnexions brutales d'utilisateurs (par exemple, si l'apprenant ferme l'onglet de son navigateur de manière inopinée), la fermeture asynchrone des

sockets réseau par les threads Rust provoquait un plantage (panic) du processus Python de l'agent avec un code d'erreur système. Cela nécessitait une intervention manuelle en console pour relancer l'agent.

- **La solution** : Nous avons développé un script de supervision et de résilience, nommé `run_agent.py`. Ce script lance le processus principal de l'agent LiveKit dans un sous-processus supervisé (sub-process). Le superviseur analyse en continu le code de retour et les flux de sortie de l'agent. Si le processus s'arrête suite à une déconnexion WebRTC ou un crash, le script de supervision intercepte la fermeture et **relance automatiquement un nouveau worker opérationnel en moins de 2 secondes**, garantissant une disponibilité permanente pour les apprenants.

![Figure 5: Schéma du cycle de résilience et d'auto-restart du superviseur] (file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1782775653290.png)

5.3. Implémentation du mécanisme de secours (fallback) STT face aux erreurs 429

- **Le problème** : L'API de transcription de Cohere Transcribe v2, bien qu'excellente pour comprendre la terminologie bancaire, impose des limites de requêtes par minute (Rate Limits) strictes sur ses clés d'évaluation. Lors des tests, le bruit de fond des pièces d'entraînement activait le VAD de LiveKit de manière répétée, envoyant de nombreuses requêtes de silence à l'API Cohere, ce qui épuisait le quota en moins de deux minutes et renvoyait une erreur réseau 429 Too Many Requests. L'agent devenait alors sourd et incapable de répondre.
- **La solution** : Nous avons réécrit la classe d'intégration STT pour intercepter les exceptions réseau. En cas de détection d'une erreur 429 de Cohere, le code bascule **automatiquement et de manière invisible** sur le moteur de secours **OpenAI Whisper STT** pour transcrire la phrase de l'utilisateur. Ce mécanisme de secours asynchrone garantit que la simulation se poursuit sans aucune interruption pour l'apprenant.

![Figure 6: Diagramme logique du fallback dynamique STT] (file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1784714274136.png)

5.4. Ingénierie de prompt avancée : Modération et identification progressive

Pour répondre aux retours des équipes de la BMCI lors de la présentation à mi-parcours, nous avons grandement affiné les consignes système du LLM :

- **Suppression des didascalies** : Les modèles de langage insèrent naturellement des indications d'actions entre astérisques (ex: `*soupir*`, `*frappe sur la table*`). Ces indications polluaient le texte affiché et perturbaient le moteur de synthèse vocale, qui lisait parfois ces expressions textuellement. Une consigne stricte de formatage a été ajoutée pour bannir tout texte non prononcé à voix haute.
- **Identification progressive du client (KYC)** : La cliente (Mme. Sarah Bennani) a interdiction de donner son nom, son numéro de carte d'identité (CIN : AB123456) ou l'historique de ses opérations (dépôt de 15 000 DH) de manière spontanée. Elle ne doit divulguer ces informations **que si le conseiller lui formule une demande d'identification polie et réglementaire**. Cette consigne permet d'entraîner

concrètement les conseillers à respecter les procédures de sécurité de la banque.

- **Humeur adaptative** : Le prompt définit un comportement dynamique en fonction de la politesse et du calme de l'apprenant. Si le conseiller est calme et à l'écoute, la cliente s'apaise et accepte un virement instantané. S'il est froid ou utilise un jargon technique rigide, la cliente s'agace et exige de parler à un responsable.

![Figure 7: Interface de discussion LiveKit avec Mme. Sarah Bennani montrant le dialogue fluide] (file:///C:/Users/user/.gemini/antigravity/brain/0aa50022-c315-4341-8510-fa64795e2544/media__1784716502765.png)

6. Analyse Comparative : L'Évolution Speech-to-Speech (Moshi) et Perspectives

6.1. Analyse des contraintes d'infrastructure et de transport de Moshi

Pour réduire encore la latence et obtenir une interaction en temps réel quasi instantanée, l'utilisation de modèles natifs de voix à voix (Speech-to-Speech ou S2S) comme **Moshi** (développé par Kyutai) a été étudiée. Cette technologie élimine la cascade STT→LLM→TTS pour descendre à une latence inférieure à **200 millisecondes**. Cependant, son déploiement industriel en entreprise se heurte à plusieurs barrières majeures :

- **Infrastructure matérielle coûteuse** : Moshi ne peut pas s'exécuter sur processeur (CPU). Il requiert au minimum de **16 Go à 24 Go de mémoire vidéo (VRAM)** dédiée et rapide. En production, cela implique l'utilisation de serveurs cloud équipés de cartes NVIDIA professionnelles (comme les A10G, A100 ou H100), ce qui représente un coût d'exploitation récurrent très important pour la banque.
 - **Développement du protocole réseau** : Moshi est fourni sous forme d'algorithme d'inférence brut. Il ne dispose pas de serveur de transport audio. Il est nécessaire de développer une couche réseau WebSocket pour capturer le micro utilisateur, compresser l'audio via le codec *Mimi*, gérer la perte de paquets et la gigue réseau (Jitter Buffer).
 - **Gestion des interruptions** : Contrairement aux systèmes STT-LLM-TTS où le silence déclenche la réponse, le modèle S2S génère de l'audio en continu. Gérer proprement les interruptions (couper la voix de l'IA dès que l'utilisateur prend la parole) nécessite de coder un protocole de purge des tampons audio côté serveur.
 - **Incompatibilité Windows** : Moshi est optimisé pour Linux et macOS. Son exécution sur des serveurs Windows requiert le passage par WSL2 (Windows Subsystem for Linux), ce qui complique l'architecture et peut réintroduire de la latence de traitement audio.
 - **Interfaçage téléphonique (VoIP / SIP)** : Pour que la simulation s'effectue via un vrai téléphone, il faut développer une passerelle VoIP/SIP connectée au standard de la BMCI pour convertir le flux téléphonique en paquets audio compatibles avec le codec Mimi de Moshi.
 - **Fonctionnalités métier (Function Calling)** : Moshi produit de la parole sans étape de texte intermédiaire exploitable. Il est complexe d'interroger une base de données bancaire en temps réel (ex: pour bloquer une carte bancaire) à partir d'un flux audio brut, ce qui nécessite l'interception et le traitement du monologue intérieur généré par le modèle en parallèle de sa voix.
-

6.2. Perspectives d'intégration RAG et classification émotionnelle acoustique

Pour faire évoluer la solution vers une version industrielle à la BMCI, deux axes majeurs de recherche ont été identifiés :

1. **Intégration d'un RAG (Retrieval-Augmented Generation) Contextuel** : Connecter l'agent à une base de données de documents internes de la BMCI (adresses d'agences, conditions tarifaires réelles, procédures de réclamation). À chaque réplique, le modèle pourra interroger cette base pour citer des adresses ou des règles bancaires réelles du Maroc, renforçant le réalisme de la simulation.
 2. **Analyse acoustique des émotions** : Actuellement, l'agent modifie son humeur en se basant sur le texte transcrit de l'utilisateur. L'objectif futur est d'intégrer un modèle de classification audio léger (ex: Wav2Vec2) pour analyser directement l'intonation physique de la voix de l'apprenant (stress, calme, irritation) et transmettre ce score d'émotion au LLM, rendant les réactions de la cliente encore plus authentiques.
-

7. Bibliographie

1. **LiveKit Inc.** - *Real-time WebRTC architecture and AI agent SDK documentation*, <https://docs.livekit.io>
2. **Mistral AI** - *Voxtral Text-to-Speech models and API endpoints specifications*, <https://docs.mistral.ai>
3. **Cohere Inc.** - *Cohere Transcribe v2: ASR model capabilities and billing guidelines*, <https://docs.cohere.com>
4. **Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I.** (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. arXiv preprint arXiv:2212.04356. (Whisper model).
5. **Hugging Face** - *Open ASR Leaderboard: Evaluation of Speech-to-Text models on French datasets*, https://huggingface.co/spaces/hf-audio/open_asr_leaderboard
6. **Kyutai** - *Moshi: A Real-Time Speech-to-Speech Dialogue Model*, <https://moshi.chat>